DaemonSets in Kubernetes

Since you're learning **KCNA as a beginner**, let's keep this very simple.

What is a DaemonSet?

A DaemonSet is a Kubernetes object that makes sure a copy of a Pod runs on every Node (or every selected Node) in the cluster.

Easy memory:

📌 **DaemonSet = One Pod on every Node**

Simple Example

Suppose your Kubernetes cluster has 3 Nodes:

Kubernetes Cluster

Node 1

└── Monitoring Pod

Node 2

└── Monitoring Pod

Node 3

└── Monitoring Pod

The DaemonSet automatically creates **one copy of the Pod on each Node**.

What happens if a New Node is Added?

Suppose you already have:

Node 1 → Pod

Node 2 → Pod

Node 3 → Pod

Now you add:

Node 4

The DaemonSet automatically creates:

Node 4 → Pod ✅

So now:

Node 1 → Pod

Node 2 → Pod

Node 3 → Pod

Node 4 → Pod

That's the main purpose of a DaemonSet.

What if a Node is Removed?

Suppose:

Node 3 → Removed ❌

Its DaemonSet Pod goes away with that Node.

The remaining Nodes continue to have their Pods.

Why Do We Use DaemonSets?

DaemonSets are useful when you need **one Pod on each Node**.

Common examples include:

1. Monitoring

You may want a monitoring agent on every Node.

Node 1 → Monitoring Agent

Node 2 → Monitoring Agent

Node 3 → Monitoring Agent

2. Log Collection

You may want a log collector running on every Node.

Node 1 → Log Agent

Node 2 → Log Agent

Node 3 → Log Agent

3. Node-level Networking

Some networking components need to run on every Node.

DaemonSet vs Deployment

This is **very important** because you've already learned Deployments.

Deployment

A Deployment manages a specified number of replicas.

Example:

replicas: 3

You might get:

Node 1 → Pod

Node 2 → Pod

Node 3 → Pod

The Pods are distributed according to scheduling rules.

DaemonSet

A DaemonSet is designed to run a Pod on **each eligible Node**.

Node 1 → Pod

Node 2 → Pod

Node 3 → Pod

Node 4 → Pod

If there are 4 eligible Nodes → approximately 4 DaemonSet Pods.

Easy Comparison

Deployment	**DaemonSet**
Runs a desired number of replicas	Runs a Pod on each eligible Node
Example: 3 replicas	Example: 1 Pod per Node
Used for applications	Often used for node-level agents
Number of Pods is explicitly controlled	Number of Pods follows eligible Nodes

---

DaemonSet Example YAML

Here's a simple example:

```yaml
apiVersion: apps/v1
kind: DaemonSet

metadata:
  name: my-daemon

spec:
  selector:
    matchLabels:
      app: my-daemon

  template:
    metadata:
      labels:
```

```yaml
      app: my-daemon

  spec:
    containers:
    - name: my-container
      image: nginx
```

Notice something important:

There is **no**:

replicas: 3

Instead, Kubernetes creates a Pod for each eligible Node.

Useful Commands

Create DaemonSet

kubectl apply -f daemonset.yaml

See DaemonSets

kubectl get daemonsets

or:

kubectl get ds

ds is the short name for DaemonSet.

See DaemonSet Details

kubectl describe daemonset my-daemon

See the Pods

kubectl get pods -o wide

The -o wide helps you see **which Node each Pod is running on**.

Delete a DaemonSet

kubectl delete daemonset my-daemon

DaemonSet Flow

```
      DaemonSet
         |
         ▼
   Check eligible Nodes
         |
   ┌─────────┼─────────┐
   ▼     ▼     ▼
  Node 1  Node 2  Node 3
   |     |     |
   ▼     ▼     ▼
   Pod    Pod    Pod
```

Add another Node:

```
Node 4
  |
  ▼
Pod automatically created
```

DaemonSet + Taints/Tolerations

Remember your previous topic?

Taints and Tolerations can affect which Nodes a DaemonSet Pod can run on.

For example, if a Node is specially tainted, the DaemonSet Pod may need an appropriate **toleration** to run there.

So:

DaemonSet

|

▼

One Pod per eligible Node

|

▼

Taints/Tolerations can affect eligibility

Important KCNA Points ⭐

Remember these:

- **DaemonSet = Pod on every eligible Node.**
- Automatically creates a Pod when a new eligible Node is added.
- Commonly used for **monitoring agents**.
- Commonly used for **log collection agents**.
- Used for **node-level services**.
- Unlike a Deployment, you normally don't specify a replica count for the DaemonSet.
- kubectl get ds → view DaemonSets.
- kubectl describe ds <name> → view DaemonSet details.

🧠 **One-Line Memory Trick**

Deployment = "I want X copies of my application."

DaemonSet = "I want one copy on every eligible Node."